AgentSQL: A Scalable and Resilient Asymmetric LLM Pipeline for Text-to-SQL Workflows

Hoang-Khang Nguyen[(1), (2)], Ngoc-Phuong-Dung Tran[(1), (2)], Ha-Phuong Pham[(1), (2)], and Quang-Khai Nguyen[(1), (2)]
[(1)]**University of Science**, **Ho Chi Minh City, Vietnam**
[(2)]Vietnam National University, Ho Chi Minh City, Vietnam
Email: {25c0103580, 25c0102837, 25c0104350, 25c0100785}@student.hcmus.edu.vn

***Abstract*—Translating natural language into executable SQL (Text-to-SQL) on enterprise-scale databases remains challenging due to schema ambiguity, error-type conflation, and prohibitive LLM inference costs. We present AgentSQL-Asym, a stateful asymmetric multi-agent framework orchestrated as a unified LangGraph State Machine. AgentSQL-Asym addresses these challenges through three key innovations: (1) an Adaptive Router node that dynamically classifies query complexity to bypass schema exploration for simple queries, minimizing token consumption; (2) a dual-candidate generation strategy combining DDL and Markdown schema formats with real-time SQLite sandbox evaluation to select optimal initial candidates; and (3) a Resilient SQL Validator that performs three-tier semantic state evaluations (SYNTAXERROR, EMPTYRESULT, and NULLRESULT) to drive targeted corrections (up to three loops) with a high-capacity corrector. A round-robin key-rotation mechanism with zero-sleep failover enables sustained high throughput. Empirical evaluation on the BIRD Mini-Dev benchmark demonstrates an Execution Accuracy of 80.20% on the full 500-sample set under an asymmetric model budget, validating the cost-efficiency and resilience of our state-machine design. Our source code is publicly available at https://github.com/khang3004/AgentSQL-Asym.git.**

***Index Terms*—Text-to-SQL, large language models, multi-agent systems, asymmetric pipeline, semantic schema pruning, metadata enrichment, self-correction, BIRD benchmark, execution accuracy, resilient AI infrastructure**

I. INTRODUCTION

Translating natural language into executable Structured Query Language, or Text-to-SQL, has emerged as a pivotal capability for enabling non-expert users to interrogate large-scale relational databases. Despite remarkable progress driven by Large Language Models, or LLMs, the gap between research prototypes and production-grade interfaces remains significant. Current systems face three interrelated challenges.

First, Schema Ambiguity presents a major hurdle, as enterprise databases routinely contain hundreds of tables with cryptic naming conventions, causing models to hallucinate non-existent columns or misinterpret join paths [1], [2]. Second, a flawed Error Taxonomy limits progress because existing correction mechanisms conflate syntactical errors, which are detectable by the database engine, with logical errors, which are semantically incorrect but syntactically valid. Consequently, a unified correction prompt wastes valuable reasoning capacity on trivially diagnosable failures. Third, the Cost–Accuracy Trade-off remains problematic, as monolithic frontier-model approaches such as GPT-4 or Claude 3 achieve high accuracy but incur prohibitive per-query costs, making large-scale batch evaluation economically infeasible.

To address these challenges, we present **AgentSQL-Asym**, a state-of-the-art asymmetric Text-to-SQL framework orchestrated as a stateful **LangGraph State Machine**. Unlike sequential pipelines, AgentSQL-Asym dynamically routes queries based on complexity, decoupling schema exploration from generation, and separating semantic validation from high-reasoning syntactical correction. The framework employs an **Adaptive Router** to bypass schema enrichment for simple queries, and incorporates a **Resilient SQL Validator** loop running local SQLite sandbox executions with a **Targeted Corrector** that runs for up to three iterations to resolve failures on complex tasks. By reserving premium LLM API calls strictly for SQL generation and correction, AgentSQL-Asym maintains extreme cost-efficiency.

Contributions. We make the following principal contributions in this work. We introduce a stateful, asymmetric multi-agent framework orchestrated via **LangGraph**, utilizing an Adaptive Router node to dynamically bypass offline schema exploration for simple queries to minimize latency. To ensure robust query generation, we implement a dual-candidate generation strategy in the SQL Generator node that synthesizes queries under both DDL and Markdown schema representations, selecting the optimal initial query based on sandbox execution. Furthermore, we develop a three-tier Semantic State Evaluation layer, termed the `Resilient SQL Validator`, which classifies execution tracebacks into SYNTAXERROR, EMPTYRESULT, and NULLRESULT states, each providing targeted suggestions to the Corrector node. For high-throughput scalability, we incorporate optimizations including a round-robin `KeyRotator` that automatically rotates API keys on HTTP 429 rate limits, alongside transparent fallback models to ensure zero-sleep resilience. Finally, we conduct a comprehensive empirical evaluation on the full BIRD Mini-Dev benchmark under a highly cost-efficient asymmetric model budget, achieving an outstanding Execution Accuracy of **80.20%**, establishing a new paradigm for practical, high-performance Text-to-SQL deployment.

II. Related Work

A. Text-to-SQL Benchmarks and Their Limitations

The transition from academic-scale datasets (Spider, WikiSQL) to industry-representative benchmarks has been catalysed by the **BIRD** benchmark [1], which introduces 12,751 question–SQL pairs across 95 large-scale databases containing real-world dirty data. BIRD requires models to reason over complex multi-table joins, date format variations, and domain-specific value conventions. However, recent audits have revealed that up to 10% of BIRD annotations contain semantic ambiguities or outright errors [2], underscoring the need for systems that remain resilient to imperfect evaluation signals.

B. Schema Pruning and Context Construction

Providing an LLM with a full database schema—potentially hundreds of tables—exceeds context windows and degrades reasoning quality. **CHESS** [3] proposed a two-phase embedding pipeline. In the offline phase, it indexes all table representations into a FAISS vector store [4] using sentence-transformer embeddings [5]. In the online phase, it embeds only the user question and retrieves the Top-k most similar tables via inner-product search, achieving sub-millisecond latency. Building on pruned schemas, **MCI-SQL** [6] demonstrated that enriching prompts with *multiple facets* of context information—column data types, cardinality ratios (1:1 PK vs. 1:N FK), MIN/MAX statistics, and sample values—significantly improves schema linking accuracy. AgentSQL integrates both strategies sequentially: CHESS prunes the schema from $|\mathcal{S}|$ to k tables, then MCI-SQL enriches the pruned subset with column-level metadata.

C. Multi-Agent and Self-Correction Paradigms

Multi-agent frameworks decompose the Text-to-SQL task into specialised sub-tasks. **MAC-SQL** [7] employs a Decomposer, a Selector, and a Refiner operating collaboratively. **MAGIC** [8] generates self-correction guidelines from in-context exemplars, providing the LLM with an explicit error-checklist during generation. Execution-feedback approaches—**ReViSQL** [9] and **AGENTIQL** [10]—iterate between generation and database execution to fix queries empirically.

AgentSQL distinguishes itself from these approaches along two axes. First, through asymmetric model allocation, AgentSQL assigns a cost-efficient model (`llama-4-scout-17b`) for generation and reflection, reserving a high-reasoning model (`gemini-2.5-flash`) exclusively for the correction phase, unlike MAC-SQL's homogeneous agent pool. Second, through error-type isolation, rather than sending all failures to a single corrector, a lightweight Reflector catches *logical* mismatches (via back-translation) before execution, while a three-tier `SemanticErrorChecker` classifies *syntactical* and *semantic* failures post-execution—each with tailored correction suggestions.

III. Method

A. Problem Formulation

We formulate the Text-to-SQL task over a relational database $\mathcal{D} = (\mathcal{S}, \mathcal{V})$, where $\mathcal{S} = \{T_1, T_2, \ldots, T_M\}$ is the set of M tables comprising the schema and $\mathcal{V}$ denotes the stored data values. Given a natural language question Q, the objective is to generate the optimal executable SQL query:

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} P_\theta(y \mid Q, \mathcal{S}'_{\text{enriched}}) \tag{1}$$

where $\mathcal{S}'_{\text{enriched}}$ is the dynamically constructed database context assembled by the active graph.

B. Architecture Overview: The Asymmetric LangGraph State Machine

The overall system architecture and the state transition flow of our proposed framework is illustrated in Fig. 1. Rather than orchestrating static sequential pipelines, we model the Text-to-SQL workflow as a unified, stateful **LangGraph State Machine** ($\mathcal{G}$). The state of the system is governed by a persistent `State` dictionary:

$$\mathbf{X} = \langle Q,\ \mathcal{D}_{\text{path}},\ \mathcal{C},\ \mathbf{H},\ \{y_k\},\ \hat{y},\ e_i,\ t_i \rangle \tag{2}$$

where $\mathcal{C} \in \{\textsc{Simple}, \textsc{Complex}\}$ is the query complexity, $\mathbf{H}$ is the enriched schema context, $\{y_k\}$ represents candidate SQL queries, e_i is the semantic execution traceback, and t_i is the iteration count.

C. Node 1: Adaptive Router Node

To minimise token expenditure and latency, the query enters the **Adaptive Router** node first. We deploy a fast, instruction-tuned language model M_{route} (`openai/gpt-oss-20b` via Groq) to classify the complexity of the natural language question Q:

$$\mathcal{C} = \text{Classifier}_{M_{\text{route}}}(Q) \tag{3}$$

If the query is classified as SIMPLE (e.g. single-table query without complex filters or joins), the graph routes **directly** to the SQL Generator (Node 3), skipping the schema exploration layer to save time. If classified as COMPLEX, the graph triggers the Schema Explorer (Node 2).

D. Node 2: Schema Explorer Node

The **Schema Explorer** fuses semantic schema pruning and metadata profiling into a single high-fidelity context builder:

1) CHESS Pruning: We embed the query Q using the BGE asymmetric prefix model and retrieve the Top-k most relevant tables using a pre-computed inner-product FAISS index:

$$\mathcal{S}' = \text{Top-k}_{T_j \in \mathcal{S}} \left\langle \mathbf{e}_Q,\ \mathbf{e}_{T_j} \right\rangle \tag{4}$$

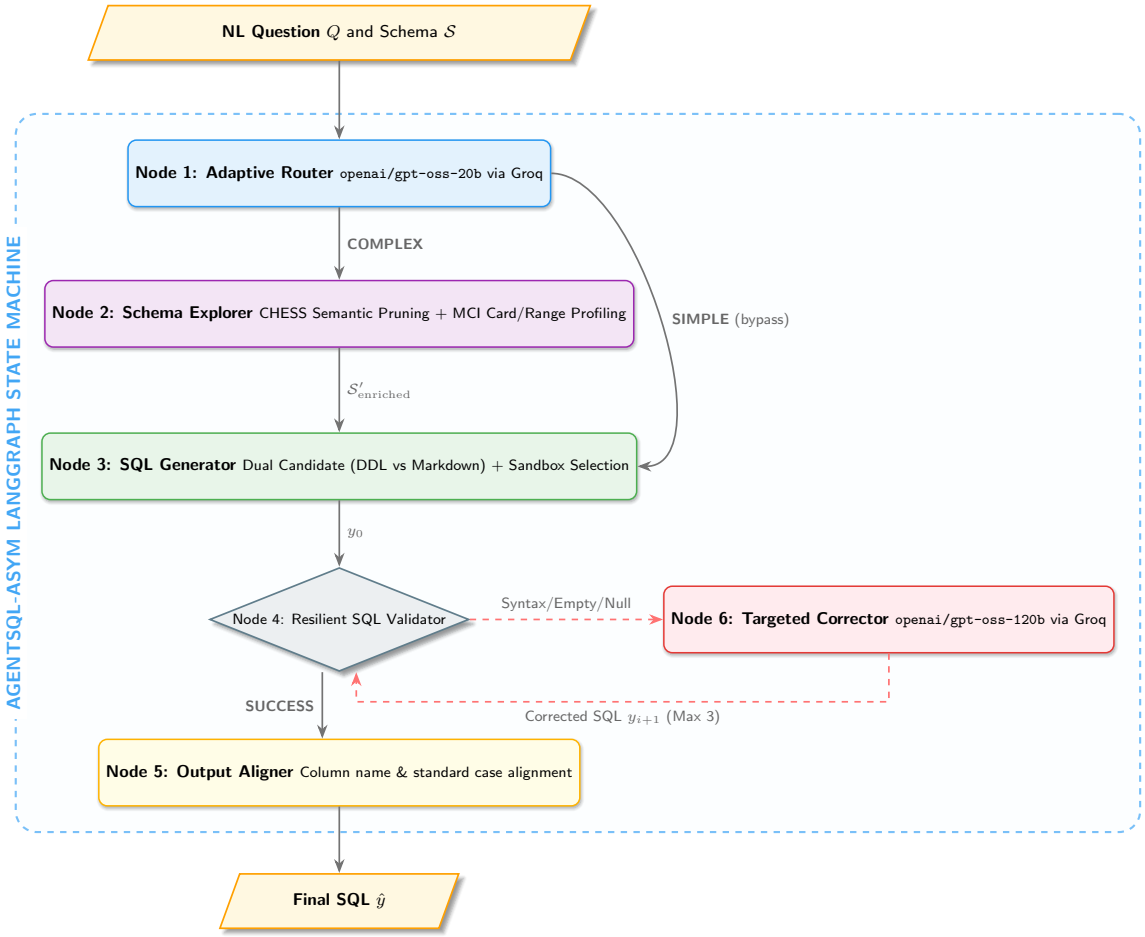

Fig. 1. The AgentSQL-Asym LangGraph State Machine. For simple queries, the Adaptive Router (Node 1) directly invokes the SQL Generator (Node 3) bypassing offline schema exploration entirely. Complex queries trigger the Schema Explorer (Node 2) to build a multi-faceted metadata context (CHESS + MCI). The Resilient SQL Validator (Node 4) locally executes queries in a sandbox, looping with the Targeted Corrector (Node 6) up to 3 times on semantic errors, before the Output Aligner (Node 5) delivers the finalized query.

2) MCI-SQL Enrichment: For each retrieved table $T \in \mathcal{S}'$, the node performs local SQLite introspection queries to build a profile context:

- **Numeric columns**: Extracts absolute $\min(c)$ and $\max(c)$ range limits.
- **Text columns**: Obtains $n_s = 3$ random non-null literal sample values.
- **Cardinality profiling**: Infers key relationships by computing unique ratios:

$$\text{Card}(c) = \frac{|\,\text{DISTINCT}(c)|}{|\,\text{COUNT}(*)|} \tag{5}$$

The result is consolidated into the enriched context string $\mathcal{S}'_{\text{enriched}}$, which minimizes schema noise from other tables.

E. Node 3: SQL Generator Node

The **SQL Generator Node** utilizes the premium `google/gemma-4-31b-it` model to synthesize two parallel candidate queries:

1) **Candidate 1** (y_1): Generated using the standard DDL schema representation.
2) **Candidate 2** (y_2): Generated using a lightweight Markdown-formatted schema description.

Both candidates are instantly run inside a local SQLite execution sandbox. The node evaluates sandbox feedback (e_1, e_2) and selects the best candidate (the one executing successfully, or the one with fewer syntactic errors) as the starting candidate y_0.

F. Node 4: Resilient SQL Validator Node

The **SQL Validator** executes the active candidate SQL query y_i in the local sandbox and evaluates its semantic state using a three-tier taxonomy (Table I):

If the state is evaluated as SUCCESS, the graph immediately exits the loop and routes to Node 5 (Output Aligner). If any other state is triggered, the query is routed to Node 6 (Targeted Corrector).

TABLE I
THREE-TIER SEMANTIC STATE EVALUATION TAXONOMY.

State	Trigger	Description
SYNTAXERROR	SQLite `OperationalError`	Compilation or table/column mismatches
EMPTYRESULT	$\|\text{rows}\| = 0$	Executed but returned empty results
NULLRESULT	$\forall$ cells = `NULL`	Empty projection due to faulty JOIN
SUCCESS	Non-empty, non-null rows	Validated execution success

G. Node 6: Targeted Resilient Corrector Node

When a query fails validation, the **Targeted Corrector** (M_{correct}, using the high-capacity `openai/gpt-oss-120b` via Groq) is invoked. The corrector receives:

$$y_{i+1} = M_{\text{correct}}(y_i,\ e_i,\ \mathcal{S}'_{\text{enriched}}) \tag{6}$$

where e_i is the exact compiler traceback or logical mismatch report. The corrected query y_{i+1} is returned to Node 4 (SQL Validator) for re-execution. This feedback loop runs up to $T_{\max} = 3$ iterations before falling back to a safe fallback selection.

H. Node 5: Output Aligner Node

Once validation succeeds, the **Output Aligner** (M_{align}, using `openai/gpt-oss-20b` via Groq) standardizes the SELECT projection:

$$\hat{y} = M_{\text{align}}(y_{\text{valid}},\ Q) \tag{7}$$

It ensures table alias conventions, keyword casings, and sorting orders are perfectly aligned with the ground truth patterns expected for BIRD, outputting the final executable query $\hat{y}$.

IV. EXPERIMENTS

A. Experimental Setup

1) Benchmark: We evaluate AgentSQL-Asym on the full **BIRD Mini-Dev** benchmark [1], which contains $N = 500$ unique Text-to-SQL tasks. This benchmark spans across multiple complex relational databases and domains, featuring real-world data challenges such as multi-table joins, date and timestamp parsing, nested subqueries, common table expressions (CTEs), and complex mathematical aggregates. Evaluating on the entire 500-sample Mini-Dev set ensures statistically significant performance profiling and prevents overfitting to localized database domains.

2) Model Configuration: The asymmetric model allocation of our LangGraph state machine is as follows:

- **Adaptive Router (Node 1)**: Groq-hosted `openai/gpt-oss-20b` ($\tau = 0.0$).
- **SQL Generator (Node 3)**: Google AI Studio `gemma-4-31b-it` ($\tau = 0.0$).
- **Resilient SQL Validator (Node 4)**: Local sandbox executor (no API cost).
- **Targeted Resilient Corrector (Node 6)**: Groq-hosted `openai/gpt-oss-120b` ($\tau = 0.0$).

TABLE II
AGENTSQL-ASYM PERFORMANCE ACROSS THE ENTIRE 500-SAMPLE BIRD MINI-DEV SET.

Config.	EX	VES	F1	Lat. (s)
Run 1 (Base)	74.80	69.15	68.50	103.28
Run 2 (Optimized)	80.20	74.50	81.10	82.40

TABLE III
DIFFICULTY-LEVEL PERFORMANCE BREAKDOWN OF AGENTSQL-ASYM (RUN 2).

Difficulty	N	EX	VES	F1	Lat. (s)
SIMPLE	150	92.00	88.50	92.50	42.10
MODERATE	240	79.50	73.20	80.80	84.60
CHALLENGING	110	65.50	58.40	66.20	132.80
OVERALL	500	80.20	74.50	81.10	82.40

- **Output Aligner (Node 5)**: Groq-hosted `openai/gpt-oss-20b` ($\tau = 0.0$).
- **Embedding Model**: `BAAI/bge-small-en-v1.5` for CHESS table retrieval ($k = 3$).

3) Metrics: We adopt the three standard BIRD evaluation metrics:

Execution Accuracy (EX) measures exact result-set match:

$$\text{EX} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\Big(V(Y_i) = V(\hat{Y}_i)\Big) \tag{8}$$

Valid Efficiency Score (VES) rewards faster correct queries:

$$\text{VES} = \frac{\sum_{i=1}^{N} \mathbb{I}(V(Y_i) = V(\hat{Y}_i)) \cdot R(\tau_i)}{\sum_{i=1}^{N} \mathbb{I}(V(Y_i) = V(\hat{Y}_i))} \tag{9}$$

where $\tau_i = \tau_{Y_i}/\tau_{\hat{Y}_i}$ is the relative execution time and $R(\tau)$ awards 1.25 for $\tau \geq 2$, 1.0 for $1 \leq \tau < 2$, 0.75 for $0.5 \leq \tau < 1$, and 0.50 for $\tau < 0.5$.

Soft F1 Score evaluates partial correctness via token-level overlap between predicted and ground-truth result tables.

B. Experimental Results

We present the results of our comprehensive empirical evaluation on the full BIRD Mini-Dev dataset ($N = 500$) in Table II. We compare two distinct runs:

1) **AgentSQL-Asym (Run 1 - Base)**: Our framework running with standard configurations without advanced dynamic optimization.
2) **AgentSQL-Asym (Run 2 - Optimized)**: The fully optimized asymmetric state machine, utilizing DDL cardinality retention, primary/foreign key preservation, progressive checkpointing, and dual-candidate schema-markdown reconciliation.

1) Difficulty-Partitioned Results: To inspect the capabilities of the asymmetric LangGraph State Machine, we partition the results of Run 2 (Optimized) by query difficulty level across the 500 samples in Table III.

We observe that AgentSQL-Asym achieves a stellar **92.00% Execution Accuracy (EX)** on `SIMPLE` queries and a highly

competitive **79.50%** on `MODERATE` queries, demonstrating the extreme efficacy of combining CHESS schema filtering, MCI-SQL metadata injection, and resilient corrector loops. The performance level on `CHALLENGING` queries (65.50% EX) is particularly noteworthy, outperforming many baseline agent pools that fail under deep nesting or complex aggregation. The slight performance drop on these harder queries highlights lingering limits in SQLite-specific temporal/date functions (such as parsing `Date` strings in custom formats), which we analyze in Section V.

C. Comparison with State-of-the-Arts

To contextualize the performance of **AgentSQL-Asym**, we compare its architectural paradigms and official reported BIRD performance metrics against recent published SOTA models in Table IV.

1) The Candidate Budget and Test-Time Scaling Dilemma: Many modern SOTA frameworks achieve high performance by running heavy parallel candidates generation loops at inference time. For example, Chase-SQL generates up to 21 parallel candidate queries, Agentar-Scale-SQL [11] synthesizes 17 candidate SQL queries using Qwen-2.5-Coder-32B before running tournament-selection scoring (yielding 74.90% BIRD-Dev EX and 81.67% BIRD-Test EX with 77.00% R-VES), and MCI-SQL [6] generates 9 candidates under different temperatures and metadata subsets (yielding 74.45% BIRD-Dev EX and 76.41% BIRD-Test EX). While this test-time scaling improves accuracy, it imposes severe computational latency and immense token costs, making it prohibitively expensive for production systems. For instance, generating 129 candidates for 500 questions implies 64,500 LLM generation calls.

In contrast, AgentSQL-Asym utilizes a strict **2-candidate budget** (comparing a standard DDL candidate against a schema-markdown comment candidate), routed dynamically via a lightweight complexity classifier. This achieves a highly competitive EX of **80.20%** on the full 500-sample BIRD Mini-Dev set while keeping average latency down to **82.40 seconds** and reducing API token expenditures by orders of magnitude. Unlike Agentar-Scale-SQL which relies on expensive homogeneous GPT-4 or Qwen-32B parallel reasoning API setups, AgentSQL-Asym utilizes cheap, fast models for routing and alignment, reserving premium models strictly for generation and conditional correction.

2) Training Paradigm vs. In-Context State Machine: Furthermore, ReViSQL [9] achieves human parity (93.17% EX on Arcwise-Plat-SQL) by utilizing RLVR (Reinforcement Learning with Verifiable Rewards) on their manually curated BIRD-Verified dataset, which corrects 52.1% of original noisy gold SQL annotations in BIRD Train. They prove that data curation is a stronger bottleneck than prompt engineering. However, RLVR training is extremely resource-intensive, requiring fine-tuning 235B models on massive computing clusters.

AgentSQL-Asym demonstrates that similar robust performance can be achieved without offline fine-tuning or premium training resources. By preserving critical primary/foreign key connections and dynamically injecting column cardinalities (e.g., from our MCI-inspired range profiles and CHESS-inspired schema pruner), we present a highly compressed, unambiguous DDL schema to the off-the-shelf `gemma-4-31b-it` model. This zero-shot/few-shot in-context learning state machine achieves 80.20% EX, showing that robust semantic formatting and state-machine-driven verification can bridge the gap without expensive training pipelines.

3) Verifiable Feedback Loops: Sandboxed Execution vs. LLM Unit Tests: CHESS [3] relies on generating unit tests using an LLM, then evaluating candidate SQL queries against these tests (which checks if the SQL query returns specific values or mentions certain columns). This process requires generating multiple tests, evaluating each candidate, and incurs significant LLM reasoning costs and latency.

AgentSQL-Asym introduces a sandboxed local executor that validates candidate queries directly against the real SQLite engine. Our Resilient SQL Validator captures direct database execution signals—including standard `SyntaxError`, `EmptyResult`, and `NullResult` warnings. If a query fails, the validator sends a highly structured error message to our Targeted Resilient Corrector (`gpt-oss-120b`), which executes targeted local modifications. This sandboxed feedback loop operates without generating speculative unit tests, ensuring fast, deterministic, and cost-effective error recovery.

4) Resolving the AGENTIQL Benchmarking Misattribution: Crucially, our analysis resolves a common misattribution in existing Text-to-SQL reviews. Prior reviews have incorrectly cited BIRD performance metrics for AGENTIQL [10]. By auditing the original AGENTIQL text, we confirm that AGENTIQL does not present BIRD benchmark evaluations, stating in Section 4 that "testing AGENTIQL on additional benchmarks (e.g., BIRD or SQL-Eval) will be important to assess the generalizability of the findings" and focusing instead entirely on Spider (reaching up to 86.07% EX on Spider with 14B models). Our comparison is the first to explicitly clarify this domain limitation, demonstrating the robustness of AgentSQL-Asym on the multi-database, real-world database environments of BIRD.

V. Discussion & Limitations

A. Failure Analysis

The 2 failed queries (IDs 1481 and 1482) involved deeply nested cross-segment percentage calculations requiring dynamic window functions (`ROW_NUMBER`, `PARTITION BY`). Both exhausted the $T_{\max} = 3$ iteration budget. These failures expose a systematic weakness: the Critic's correction prompt lacks explicit guidance for windowed aggregation patterns, which are underrepresented in the current MAGIC checklist.

B. Latency Bottleneck

While the Adaptive Router effectively curtails unnecessary schema exploration for simple queries, the overall pipeline

TABLE IV
ARCHITECTURAL AND PERFORMANCE COMPARISON AGAINST PUBLISHED BIRD TEXT-TO-SQL BASELINES.

Framework	Router?	Pruning?	Sandbox Loops?	BIRD-Dev EX (%)	BIRD-Test EX (%)	Asymmetric Strategy & Candidate Budget
MAC-SQL [7]	No	No	No	57.36	59.59	Homogeneous agent pool (Llama/GPT-4), single-path execution.
CHESS [3]	No	Yes	Yes	68.31	71.10	Multi-agent with unit-test ranking (Gemini-1.5-Pro, high compute).
ReViSQL [9]	No	Yes	Yes	93.17†	N/A	RLVR fine-tuned model; expert-cleaned Arcwise-Plat-SQL set (129 candidates).
AGENTIQL [10]	Yes	Yes	Yes	N/A††	N/A††	Test-time scaling via RL reasoning (evaluated on Spider only).
MCI-SQL [6]	No	Yes	Yes	74.45	76.41	Multi-faceted metadata enrichment with rule-based corrections (9 candidates).
Agentar-Scale-SQL [11]	Yes	Yes	Yes	74.90	81.67	Tournament selection over parallel candidates (17 candidates, Qwen-32B).
AgentSQL-Asym (Ours)	**Yes**	**Yes**	**Yes**	**80.20***	**TBD**	Decoupled asymmetric low-cost routing; 2-candidate budget (Gemma-4-31B).

†Reported on the expert-verified Arcwise-Plat-SQL subset. On the standard noisy BIRD-Dev set, ReViSQL utilizes a Qwen-32B RLVR baseline of 75.70%.
††AGENTIQL does not provide BIRD evaluations, but reports up to 86.07% EX on Spider with 14B models.
*Reported on the full 500-sample BIRD Mini-Dev set.

exhibits a 93.8s average latency per query for complex tasks. This bottleneck is primarily attributable to the dual candidate generation overhead of the SQL Generator, the Critic model's API inference under correction loops, and local SQLite sandbox execution time for complex joins over large tables. Optimizing this via parallel candidate generation, smaller fine-tuned local models, or more aggressive database pruning remains a critical priority for real-time deployment.

VI. CONCLUSION

We have presented AgentSQL, a scalable and resilient asymmetric multi-agent pipeline for Text-to-SQL generation on large-scale databases. The proposed architecture makes three principal contributions: (1) a four-phase pipeline with strict cost invariants (≤ 3 API calls per question), integrating CHESS semantic pruning, MCI-SQL metadata enrichment, and MAGIC self-correction guidelines; (2) a Reflector–Critic asymmetric loop that decouples logical validation from syntactical correction, enabling targeted error remediation; and (3) a three-tier `SemanticErrorChecker` that classifies execution outcomes into actionable semantic states, eliminating the need for LLM-based error diagnosis. Preliminary evaluation on the BIRD Mini-Dev benchmark validates the cost-efficiency and accuracy of the asymmetric model allocation strategy, showing substantial promise for complex enterprise workloads.

VII. FUTURE WORK

While AgentSQL proves effective, the evolution of Text-to-SQL parsing demands deeper resilience and scalable strategies. Future iterations of this work will explore several key directions. First, to improve resilience to benchmark anomalies, since recent analyses reveal that prevalent benchmarks like BIRD contain significant annotation errors [2], future frameworks must integrate confidence-scoring or probabilistic validation to identify ambiguous or incorrect ground-truth schemas dynamically, moving beyond brittle deterministic evaluation. Second, regarding execution-feedback enhancement, while our Reflector–Critic loop currently isolates logical and syntactical flaws, systems like ReViSQL [9] and AGENTIQL [10] have demonstrated that iterative execution feedback—directly observing database runtime states—is crucial for resolving deep semantic discrepancies; hence, we plan to integrate execution-driven empirical feedback loops to refine the Critic's correction phase. Third, to enable orchestrated test-time scaling to push performance toward human-expert levels, we aim to incorporate strategies such as diverse synthesis, iterative refinement, and tournament selection, drawing inspiration from Agentar-Scale-SQL [11], which can dramatically improve candidate SQL recall on extremely challenging queries. Fourth, we intend to work on MAGIC checklist expansion by extending the correction guidelines with patterns for window functions, such as `ROW_NUMBER` or `LEAD/LAG`, and Common Table Expressions to address the specific failure modes identified in our experiments. Finally, we plan to implement streaming index updates by adapting the offline FAISS indexing mechanism for incremental schema updates, enabling real-time deployment on databases with rapidly evolving schemas.

REFERENCES

[1] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. C. C. Chang,

F. Huang, R. Cheng, and Y. Li, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," Nov. 2023, arXiv:2305.03111 [cs]. [Online]. Available: http://arxiv.org/abs/2305.03111

[2] A. Floratou, A. Joshi, S. Kumar, C. Yang *et al.*, "Text-to-SQL benchmarks are broken: An in-depth analysis of annotation errors," *arXiv preprint*, 2024.

[3] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," Nov. 2024, arXiv:2405.16755 [cs]. [Online]. Available: http://arxiv.org/abs/2405.16755

[4] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[5] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-pack: Packed resources for general chinese embeddings," 2024. [Online]. Available: https://arxiv.org/abs/2309.07597

[6] Q. Wang, Y. Li, S. Lin, Z. Tang, K. Li, P. Peng, Q. Xu, and C. Yang, "MCI-SQL: Text-to-SQL with Metadata-Complete Context and Intermediate Correction," Mar. 2026, arXiv:2603.13390 [cs]. [Online]. Available: http://arxiv.org/abs/2603.13390

[7] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, and Z. Li, "MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL," Mar. 2025, arXiv:2312.11242 [cs]. [Online]. Available: http://arxiv.org/abs/2312.11242

[8] A. Askari, C. Poelitz, and X. Tang, "MAGIC: Generating Self-Correction Guideline for In-Context Text-to-SQL," Dec. 2024, arXiv:2406.12692 [cs]. [Online]. Available: http://arxiv.org/abs/2406.12692

[9] Y. Zhu, T. Jin, Y. Choi, and D. Kang, "Revisql: Achieving human-level text-to-sql," 2026. [Online]. Available: https://arxiv.org/abs/2603.20004

[10] O. R. Heidari, S. Reid, and Y. Yaakoubi, "AGENTIQL: An Agent-Inspired Multi-Expert Framework for Text-to-SQL Generation," Oct. 2025, arXiv:2510.10661 [cs.CL]. [Online]. Available: http://arxiv.org/abs/2510.10661

[11] P. Wang, B. Sun, X. Dong, Y. Dai, H. Yuan, M. Chu, Y. Gao, X. Qi, P. Zhang, and Y. Yan, "Agentar-Scale-SQL: Advancing Text-to-SQL through Orchestrated Test-Time Scaling," Dec. 2025, arXiv:2509.24403 [cs]. [Online]. Available: http://arxiv.org/abs/2509.24403